

```
"""
RAG Document Chatbot (Google Gemini + LangChain
+ ChromaDB)
-----
-----
Upload PDF / TXT / DOCX documents, store their
embeddings in a local
ChromaDB vector store, and chat with your
documents using Google's
Gemini models via LangChain.

Setup:
1. pip install -r requirements.txt
2. Create a .env file with:
GOOGLE_API_KEY=your_key_here
3. Run: streamlit run rag_chatbot.py
"""

import os
import tempfile

import streamlit as st
from dotenv import load_dotenv

from langchain_google_genai import
ChatGoogleGenerativeAI,
GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import
Chroma
from langchain_community.document_loaders import
PyPDFLoader, TextLoader, Docx2txtLoader
from langchain.text_splitter import
RecursiveCharacterTextSplitter
from langchain.chains import RetrievalQA
```

```

# -----
# -----
# Setup
# -----
# -----

load_dotenv()

GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY")

st.set_page_config(page_title="RAG Document
Chatbot", page_icon="📄", layout="wide")
st.title("📄 Chat with your Documents")

if not GOOGLE_API_KEY:
    st.error("GOOGLE_API_KEY not found. Please
add it to your .env file.")
    st.stop()

CHROMA_DIR = "chroma_db"
os.makedirs(CHROMA_DIR, exist_ok=True)

embeddings = GoogleGenerativeAIEmbeddings(
    model="models/embedding-001",
    google_api_key=GOOGLE_API_KEY,
)

llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=GOOGLE_API_KEY,
    temperature=0.2,
)

vectordb = Chroma(
    persist_directory=CHROMA_DIR,
    embedding_function=embeddings,
)

```

```

# -----
# -----
# Helper functions
# -----
# -----

def load_document(uploaded_file):
    """Save the uploaded file to a temp path and
    load it with the right loader."""
    suffix = "." + uploaded_file.name.split(".")
    [-1].lower()

    with
tempfile.NamedTemporaryFile(delete=False,
suffix=suffix) as tmp:
        tmp.write(uploaded_file.read())
        tmp_path = tmp.name

    if suffix == ".pdf":
        loader = PyPDFLoader(tmp_path)
    elif suffix == ".docx":
        loader = Docx2txtLoader(tmp_path)
    elif suffix == ".txt":
        loader = TextLoader(tmp_path,
encoding="utf-8")
    else:
        os.unlink(tmp_path)
        raise ValueError(f"Unsupported file
type: {suffix}")

    docs = loader.load()
    os.unlink(tmp_path)

    for d in docs:
        d.metadata["source"] =
uploaded_file.name

```

```

        return docs

def add_documents_to_store(uploaded_files):
    splitter =
RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=150)
    all_chunks = []

    for f in uploaded_files:
        docs = load_document(f)
        chunks = splitter.split_documents(docs)
        all_chunks.extend(chunks)

    if all_chunks:
        vectordb.add_documents(all_chunks)
        vectordb.persist()

    return len(all_chunks)

def get_qa_chain():
    retriever =
vectordb.as_retriever(search_kwargs={"k": 4})
    return RetrievalQA.from_chain_type(
        llm=llm,
        retriever=retriever,
        return_source_documents=True,
    )

# -----
# -----
# Sidebar: Upload documents
# -----
# -----
with st.sidebar:

```

```

st.header("📁 Upload Documents")
uploaded_files = st.file_uploader(
    "Upload PDF, DOCX, or TXT files",
    type=["pdf", "docx", "txt"],
    accept_multiple_files=True,
)

if uploaded_files and st.button("Process Documents"):
    with st.spinner("Processing documents..."):
        total_chunks = add_documents_to_store(uploaded_files)
        st.success(f"Added {total_chunks} chunks from {len(uploaded_files)} file(s).")

    st.divider()
    try:
        doc_count = vectordb._collection.count()
    except Exception:
        doc_count = 0
    st.caption(f"📊 {doc_count} chunks currently stored.")

# -----
# Main chat interface
# -----

if "messages" not in st.session_state:
    st.session_state.messages = []

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

user_query = st.chat_input("Ask a question about")

```

```

your documents...")

if user_query:
    st.session_state.messages.append({"role":
"user", "content": user_query})
    with st.chat_message("user"):
        st.markdown(user_query)

    try:
        has_docs = vectordb._collection.count()
> 0
    except Exception:
        has_docs = False

    if not has_docs:
        answer = "Please upload and process at
least one document first."
    else:
        with st.spinner("Thinking..."):
            qa_chain = get_qa_chain()
            result = qa_chain.invoke({"query":
user_query})
            answer = result["result"]

    with st.chat_message("assistant"):
        st.markdown(answer)

    st.session_state.messages.append({"role":
"assistant", "content": answer})

```